

Prompt Toolkit v2

Advanced AI Prompt Engineering System

Comprehensive Project Documentation

Transformation from Proof of Concept to Production-Grade Application

Original Project: prompt-system (PoC)

Developed by: Prompt Toolkit Team

License: MIT

Python 3.8+

Table of Contents

1. Executive Summary
2. Project Overview & Original State
3. Development Phases
4. Architecture & Package Structure
5. Multi-Dimensional Prompt Analyzer
6. Smart Improvement Engine
7. AI Provider Integration Layer
8. Professional Desktop GUI
9. Prompt Library System
10. Comprehensive Metrics System
11. Configuration Management
12. Testing & Quality Assurance
13. Backward Compatibility
14. Files Added / Modified
15. Usage Guide
16. Future Development Roadmap

1. Executive Summary

Prompt Toolkit v2 is a complete overhaul of the original prompt-system Proof of Concept. The project has been transformed from a simple collection of markdown templates and basic Python scripts into a professional-grade, production-ready application for prompt engineering.

Key achievements of this transformation include: multi-dimensional prompt analysis replacing simple regex checks, a unified AI provider interface (OpenAI, Gemini, Ollama), a professional desktop GUI application, a fully searchable prompt library with persistence, and a comprehensive test suite with 69 passing tests.

The architecture follows Clean Architecture principles with clear separation of concerns, making the system extensible, maintainable, and suitable for both academic research and practical production use.

Project Statistics

Total Files:	49 (from 20)
Python Files (New):	21
Tests:	69 (all passing)
Scoring Dimensions:	8
AI Providers:	3 (OpenAI, Gemini, Ollama)

2. Project Overview & Original State

2.1 Original Project Structure (Before)

The original project was a Proof of Concept with the following structure:

```
prompt-system/  
|-- README.md  
|-- prompts/  
|   |-- 00-interactive-prompt-engineer.md  
|   |-- legacy/ (10 markdown templates)  
|-- tools/  
|   |-- prompt_analyzer.py      (75 lines, regex-only)  
|   |-- token_optimizer.py      (51 lines, tiktoken)  
|   |-- interactive_cli.py      (96 lines, basic questions)  
|   |-- requirements.txt        (1 dep: tiktoken)  
|-- LICENSE  
|-- .gitignore
```

2.2 Original Limitations Identified

- prompt_analyzer.py: Only 6 regex checks, no real quality measurement, no sub-scores
- No AI integration whatsoever - could not call any LLM API
- No graphical user interface - CLI only
- No test files
- No proper Python package structure (loose scripts)
- No database or persistence system
- Only one dependency (tiktoken)
- No type hints or documentation
- No error handling for edge cases
- No configuration management

2.3 Transformation Goals

- Replace regex-only analysis with multi-dimensional scoring (8 dimensions)
- Add smart improvement suggestions and automatic prompt rewriting
- Create unified AI provider interface (OpenAI, Gemini, Ollama)
- Build professional desktop GUI with modern dark theme
- Implement prompt library with categories, search, and persistence
- Add comprehensive metrics (tokens, readability, complexity)
- Achieve 60+ passing unit and integration tests
- Apply Clean Architecture with separation of concerns
- Maintain full backward compatibility with legacy CLI tools
- Add complete documentation for all classes and functions

3. Development Phases

Phase 1: Package Structure & Clean Architecture

Created `prompt_toolkit/` package with 9 submodules following Clean Architecture. Implemented config management with JSON persistence. Established clear separation between analysis logic, providers, library, CLI, and GUI layers.

Phase 2: Multi-Dimensional Prompt Analyzer

Built 8 scoring dimensions: Clarity, Role, Task, Context, Constraints, Examples, Output Format, Detail Level. Each dimension produces weighted scores (0.0-1.0) with supporting evidence and improvement suggestions. Full bilingual support (Arabic/English).

Phase 3: Smart Improvement Engine

Created suggestion engine that generates targeted, priority-ranked improvements. Built prompt rewriter that restructures weak prompts into professional formats. Supports both template-based and AI-assisted rewriting.

Phase 4: AI Provider Integration

Designed abstract base provider with factory pattern. Implemented OpenAI (GPT-4/3.5), Google Gemini (Gemini Pro), and Ollama (local models) providers. Each provider supports analyze, improve, generate, and availability check operations.

Phase 5: Desktop GUI Application

Built professional tkinter GUI with modern dark theme. Includes text editor with line numbers, real-time analysis with progress bars, metrics display, library browser, and settings panel. Supports keyboard shortcuts and multi-threaded analysis.

Phase 6: Prompt Library System

Implemented JSON-based persistent storage with full CRUD operations. 8 built-in prompt templates across categories. Full-text search, category filtering, and tag-based search. Bilingual support with language detection.

Phase 7: Comprehensive Testing

69 unit/integration tests covering all core modules. Tests for dimensions, metrics, suggestions, rewriter, library storage, config management, and edge cases. Temporary file and path isolation for test independence.

Phase 8: Documentation & Legacy Support

Updated README with comprehensive documentation. Created `setup.py` and `pyproject.toml` for pip installation. Legacy CLI tools updated to use new engine while maintaining identical command-line interfaces.

4. Architecture & Package Structure

4.1 Clean Architecture Layers

The project follows Clean Architecture principles with three main layers:

1. Domain Layer: Analysis engines, dimension evaluators, metrics computation - pure Python with no external dependencies.
2. Application Layer: Orchestration, suggestion generation, prompt rewriting - coordinates domain objects to deliver features.
3. Infrastructure Layer: GUI, CLI, AI providers, file persistence - handles user interaction and external systems.

4.2 Complete Directory Structure

```
prompt-toolkit/
|-- prompt_toolkit/           # Main package
|   |-- __init__.py           # Version metadata
|   |-- config.py             # JSON config management
|   |-- analyzer/             # Analysis engine
|   |   |-- engine.py         # Core orchestrator
|   |   |-- dimensions.py     # 8 scoring dimensions
|   |   |-- metrics.py        # Metrics computation
|   |-- improver/             # Improvement engine
|   |   |-- suggestions.py    # Smart suggestions
|   |   |-- rewriter.py       # Prompt rewriting
|   |-- providers/            # AI provider layer
|   |   |-- base.py           # Abstract base
|   |   |-- factory.py        # Provider factory
|   |   |-- openai_provider.py # OpenAI integration
|   |   |-- gemini_provider.py # Gemini integration
|   |   |-- ollama_provider.py # Ollama integration
|   |-- library/              # Prompt library
|   |   |-- storage.py        # JSON persistence + CRUD
|   |   |-- catalog.py        # 8 built-in templates
|   |-- cli/                  # CLI interfaces
|   |   |-- analyze.py        # Analysis CLI
|   |   |-- interactive.py    # Interactive builder
|   |-- gui/                  # Desktop GUI
|   |   |-- app.py            # Main window
|   |   |-- editor.py         # Text editor widget
|   |   |-- analyzer_panel.py # Results panel
|   |   |-- library_panel.py  # Library browser
|-- tests/                    # Test suite
|   |-- test_analyzer.py      # 45 tests
|   |-- test_library.py       # 16 tests
|   |-- test_config.py        # 7 tests
|-- tools/                    # Legacy CLI (backward compat)
|   |-- prompt_analyzer.py    # Updated wrapper
|   |-- token_optimizer.py    # Updated wrapper
|   |-- interactive_cli.py    # Updated wrapper
```

```
|-- prompts/           # Markdown templates
|-- setup.py / pyproject.toml # Package config
|-- requirements.txt    # Dependencies
|-- README.md          # Documentation
```

5. Multi-Dimensional Prompt Analyzer

5.1 Eight Scoring Dimensions

The new analyzer evaluates prompts across 8 quality dimensions, each with configurable weights and bilingual pattern detection (Arabic/English):

Clarity (1.0x)

Detects vague vs specific language. Analyzes word choice, identifies placeholder terms (thing, stuff, etc.), measures precision of vocabulary.

Role (1.2x)

Checks for clear AI persona definition. Recognizes Arabic patterns (khaleel, bisefatak) and English patterns (act as, you are, role:).

Task (1.5x)

Evaluates task definition quality. Detects numbered steps, bullet points, and clear objective statements. Highest weighted dimension.

Context (1.0x)

Measures background information richness. Counts context clues, evaluates explanatory content, checks for situational framing.

Constraints (1.3x)

Counts explicit rules, boundaries, and limitations. Tracks categories: avoidance, requirements, format. Second highest weight.

Examples (0.8x)

Detects illustrative examples and demonstrations. Uses word-boundary-aware patterns to avoid false positives.

Output Format (1.2x)

Identifies format specifications. Recognizes table, JSON, markdown, list, code, and report formats.

Detail Level (1.0x)

Evaluates prompt length and sentence structure. Uses word count thresholds and average sentence length analysis.

5.2 Scoring Algorithm

Each dimension produces a normalized score (0.0 to 1.0). The overall score is computed as a weighted average: $\text{overall} = \frac{\sum(\text{score} * \text{weight})}{\sum(\text{weight})} * 100$. This ensures important dimensions (Task, Constraints) have proportionally higher impact. The final score is capped at [0, 100] and classified into quality tiers.

5.3 Quality Tiers

Score Range	Description
90-100: Excellent	Ready for use with minor tweaks
75-89: Very Good	Needs some improvements
60-74: Acceptable	Needs significant improvements
40-59: Weak	Needs restructuring
0-39: Poor	Must be rewritten entirely

6. Smart Improvement Engine

6.1 Suggestion Engine

The suggestion engine (suggestions.py) generates targeted, contextual improvements based on analysis results. Each suggestion includes: category, issue description, specific recommendation, optional example, and priority level (high/medium/low).

Priority classification: High-priority issues (score < 0.3 in any dimension) trigger critical suggestions with templates like 'Add a clear role'. Medium-priority issues (score 0.3-0.7) provide refinement suggestions like 'Be more specific about the role'.

6.2 Prompt Rewriter

The rewriter (rewriter.py) can restructure weak prompts into professional format. It operates in two modes:

1. Template Mode: Extracts key components and rebuilds in a structured template with Role, Task, Context, Constraints, and Output Format sections.
2. Expand Mode: For very short prompts (< 3 lines), generates a complete structured prompt around the core request.

The rewriter automatically detects the prompt language and uses appropriate Arabic or English section headers and templates.

7. AI Provider Integration Layer

7.1 Unified Interface

All AI providers implement the same `AIProvider` abstract base class with four core methods:

- `analyze_prompt()`: AI-powered prompt quality assessment
- `improve_prompt()`: Intelligent prompt rewriting
- `generate_response()`: General text generation
- `is_available()`: Connection and authentication check

7.2 Supported Providers

OpenAI (GPT-4, GPT-3.5)

Full API integration via `openai>=1.0.0`. Supports chat completions endpoint. Configurable model selection, temperature, and max tokens. Requires API key.

Google Gemini (Gemini Pro)

Integration via `google-generativeai>=0.3.0`. Uses `GenerativeModel` API. Supports same operations as OpenAI. Requires API key.

Ollama (Local LLMs)

Connects to local Ollama API endpoint (default: `localhost:11434`). Supports any locally-hosted model (Llama 2, Mistral, CodeLlama, etc.). No API key needed. Uses `urllib` for HTTP communication.

7.3 Provider Factory

The factory (`factory.py`) provides auto-detection of configured providers. It checks environment variables and config file settings. The `create_provider()` function returns the first available provider or a specific provider by name. `get_available_providers()` lists all working providers.

8. Professional Desktop GUI

8.1 Application Overview

The GUI application (`gui/app.py`) is built with `tkinter/ttk` and provides a modern, professional interface with a dark color scheme. The main window features a horizontal paned layout with the editor on the left and analysis panel on the right.

8.2 Key Components

- Editor (`editor.py`): Text editor with line numbers, syntax colors (Consolas font, dark theme bg `#1e1e2e`), undo/redo support, and scroll sync.
- Analyzer Panel (`analyzer_panel.py`): Displays overall score with color coding (green ≥ 90 , yellow ≥ 70 , orange ≥ 40 , red < 40), dimension progress bars, metrics, and suggestions.
- Library Panel (`library_panel.py`): Treeview-based browser with search bar, category filter, and CRUD operations. Double-click loads prompt into editor.
- Settings Dialog: Configuration for API keys, model selection, and provider endpoints.
- Menu Bar: File operations (Open, Save), Tools (Analyze, Improve, Rewrite), Help (About, Quick Guide).
- Keyboard Shortcuts: `Ctrl+O` (Open), `Ctrl+S` (Save), `Ctrl+R` (Analyze), `Ctrl+I` (Improve), `Ctrl+Shift+R` (Rewrite).

8.3 Threading

All analysis, improvement, and rewriting operations run in background threads to keep the GUI responsive. Results are dispatched back to the main thread via `root.after()` callbacks.

9. Prompt Library System

9.1 Storage Architecture

The library uses JSON file storage (`~/.prompt_toolkit/library.json`) for persistence. Each prompt entry is stored with: unique ID (8-char UUID), title, content, category, tags, language, score, favorite flag, and timestamps.

9.2 Features

- Full CRUD: `add()`, `get()`, `update()`, `delete()` operations with auto-persistence
- Search: Full-text search across title and content with query matching
- Filtering: Category filter and tag-based search (matches any specified tag)
- Categories: 10 predefined categories (general, coding, writing, marketing, analysis, education, business, creative, technical, other)
- Built-in Templates: 8 professionally crafted prompts seeded on first use
- Language Detection: Automatic language tagging during import
- Sorting: Results sorted by last update time (newest first)

9.3 Built-in Prompt Templates

- Code Review Prompt - Senior engineer code review checklist
- Academic Writing Assistant - Arabic academic paper helper
- Marketing Strategy Generator - Comprehensive marketing plan
- Data Analysis Prompt - Analytics and insight extraction
- UI/UX Designer - Arabic interface design specification
- Lesson Plan Creator - Educational curriculum planning
- Business Plan Generator - Startup business plan with financials
- Technical Documentation Writer - API and system documentation

10. Comprehensive Metrics System

10.1 Available Metrics

Metric	Description
Word Count	Total words in the prompt
Character Count	Total characters (with and without spaces)
Sentence Count	Estimated sentences using punctuation-aware splitting
Average Word Length	Mean characters per word
Average Sentence Length	Mean words per sentence
Token Estimates	GPT-4, Claude, and Gemini token counts using character-based heuristics
Reading Time	Estimated reading time in seconds/minutes (200 wpm)
Complexity Score	0-1 score based on vocabulary richness, sentence complexity, and word complexity

10.2 Token Estimation Algorithm

Since tiktoken is optional, the metrics system uses character-based heuristics: Arabic characters are estimated at 1.5 chars/token, English at 4 chars/token, and other characters at 3 chars/token for GPT-4. Claude uses 3.5 chars/token, Gemini uses 3 chars/token. When tiktoken is available, it is used for exact counts.

10.3 Language Detection

The detect_language() function analyzes Unicode character ranges to determine if a prompt is Arabic (>80% Arabic chars), English (>80% Latin), mixed, or unknown. This drives bilingual template selection and suggestion language.

11. Configuration Management

11.1 Config File Location

Configuration is stored in `~/.prompt_toolkit/config.json` as a JSON file.

11.2 Configuration Options

```
{
  "openai_api_key": "",           # OpenAI API key
  "openai_model": "gpt-4",       # OpenAI model name
  "gemini_api_key": "",          # Google Gemini API key
  "gemini_model": "gemini-pro",  # Gemini model name
  "ollama_endpoint": "http://localhost:11434", # Ollama URL
  "ollama_model": "llama2",      # Ollama model name
  "default_provider": "none",    # Auto-detect if "none"
  "language": "auto",           # Auto-detect prompt language
  "token_limit_warning": 4000,   # Warning threshold
  "theme": "default"            # GUI theme
}
```

11.3 Environment Variables

API keys can also be set via environment variables:

- OPENAI_API_KEY: Overrides config file value for OpenAI
- GEMINI_API_KEY: Overrides config file value for Gemini

Environment variables take precedence over config file values.

12. Testing & Quality Assurance

12.1 Test Coverage

The project includes 69 tests across 3 test files covering all core modules.

test_analyzer.py (45 tests)

Dimensions, metrics, language detection, full analysis pipeline, suggestions, rewriter. Tests for all 8 dimensions (found/missing/edge cases), metrics edge cases (empty text, Arabic text, long text), and improvement plan structure.

test_library.py (16 tests)

Entry creation, serialization, CRUD operations, search by query/category, category distribution, persistence across instances, edge cases (empty library, nonexistent entries).

test_config.py (7 tests)

Default values, save/load cycle, merge behavior, directory creation, unknown key filtering.

12.2 Running Tests

```
# Run all tests
python -m unittest discover tests -v

# Run specific test file
python -m unittest tests.test_analyzer -v
python -m unittest tests.test_library -v
python -m unittest tests.test_config -v
```


13. Backward Compatibility

13.1 Legacy Tool Wrappers

The original CLI tools in `tools/` have been updated to use the new engine while maintaining identical command-line interfaces. Existing scripts and workflows continue to work without modification:

```
# Old commands (still work the same way)
cd tools
python prompt_analyzer.py prompt.md      # Same CLI args
python token_optimizer.py prompt.md      # Same CLI args
python interactive_cli.py                 # Same user flow
```

13.2 New Capabilities via Old Tools

Users of legacy tools automatically benefit from the new multi-dimensional analysis engine without changing their workflow. The legacy `prompt_analyzer` now provides 8 dimension scores, metrics, and more accurate suggestions.

14. Files Added / Modified

14.1 New Files Created (21 files)

- prompt_toolkit/__init__.py
- prompt_toolkit/config.py
- prompt_toolkit/analyzer/__init__.py
- prompt_toolkit/analyzer/engine.py
- prompt_toolkit/analyzer/dimensions.py
- prompt_toolkit/analyzer/metrics.py
- prompt_toolkit/improver/__init__.py
- prompt_toolkit/improver/suggestions.py
- prompt_toolkit/improver/rewriter.py
- prompt_toolkit/providers/__init__.py
- prompt_toolkit/providers/base.py
- prompt_toolkit/providers/factory.py
- prompt_toolkit/providers/openai_provider.py
- prompt_toolkit/providers/gemini_provider.py
- prompt_toolkit/providers/ollama_provider.py
- prompt_toolkit/library/__init__.py
- prompt_toolkit/library/storage.py
- prompt_toolkit/library/catalog.py
- prompt_toolkit/cli/__init__.py
- prompt_toolkit/cli/analyze.py
- prompt_toolkit/cli/interactive.py
- prompt_toolkit/gui/__init__.py
- prompt_toolkit/gui/app.py
- prompt_toolkit/gui/editor.py
- prompt_toolkit/gui/analyzer_panel.py
- prompt_toolkit/gui/library_panel.py
- tests/__init__.py
- tests/test_analyzer.py
- tests/test_library.py
- tests/test_config.py
- setup.py
- pyproject.toml
- requirements.txt

14.2 Modified Files (5 files)

- README.md (complete rewrite with v2 documentation)
- tools/prompt_analyzer.py (updated to use new engine)
- tools/token_optimizer.py (updated to use new metrics)
- tools/interactive_cli.py (delegates to new interactive CLI)
- tools/requirements.txt (updated with optional deps)

14.3 Unchanged Files (11 files)

- .gitattributes
- .gitignore
- LICENSE
- prompts/00-interactive-prompt-engineer.md
- prompts/legacy/ (10 markdown files: 01 through 10)

15. Usage Guide

15.1 Installation

```
# Install from source
pip install -e .

# With optional AI provider support
pip install -e ".[all]"

# Or specific providers
pip install -e ".[openai]"
pip install -e ".[gemini]"
```

15.2 Quick Start Examples

```
# 1. Launch GUI (Desktop Application)
python -m prompt_toolkit.gui.app
# OR: prompt-gui

# 2. Analyze a prompt file
python -m prompt_toolkit.cli.analyze prompt.md
python -m prompt_toolkit.cli.analyze prompt.md --json
# OR: prompt-analyze

# 3. Interactive prompt builder
python -m prompt_toolkit.cli.interactive
# OR: prompt-interactive
```

15.3 Python API Examples

```
# Analyze a prompt programmatically
from prompt_toolkit.analyzer.engine import PromptAnalyzer
analyzer = PromptAnalyzer()
report = analyzer.analyze("Write Python code")
print(f"Score: {report.overall_score:.1f}/100")
print(f"Suggestions: {len(report.all_suggestions)}")

# Improve a prompt
from prompt_toolkit.improver.rewriter import PromptRewriter
rewriter = PromptRewriter()
improved = rewriter.rewrite_template("Write code")

# Use AI provider
from prompt_toolkit.providers.factory import create_provider
provider = create_provider("openai")
if provider and provider.is_available():
    result = provider.analyze_prompt("Your prompt")

# Manage prompt library
from prompt_toolkit.library.storage import PromptLibrary, PromptEntry
library = PromptLibrary()
entry = PromptEntry(title="My Prompt", content="...", category="coding")
library.add(entry)
```

```
results = library.search(query="python")
```

16. Future Development Roadmap

Web Interface

Build React-based web frontend with FastAPI backend for cloud-based access, user authentication, and collaborative prompt engineering.

Plugin System

Create a plugin architecture for custom scoring dimensions, analysis rules, and export formats. Users can contribute community plugins.

Export Reports

Add PDF/DOCX/HTML report generation for analysis results. Include visual charts, detailed breakdowns, and improvement tracking over time.

Batch Analysis

Support analyzing multiple prompts in batch mode with aggregated statistics and comparison views for A/B testing.

Version History

Track changes to prompts over time with diff views, rollback support, and change attribution.

Cloud Sync

Enable sharing prompt libraries across devices via cloud providers, with team collaboration features.

More AI Providers

Add Anthropic Claude, Cohere, Hugging Face Inference API, and Azure OpenAI providers.

Advanced Readability

Implement Flesch-Kincaid for English and custom Arabic readability indices. Add grade-level estimation.

Prompt Marketplace

Community-driven platform for sharing and discovering prompt templates, with ratings and usage statistics.

CI/CD Integration

GitHub Actions for automated testing, linting, and publishing to PyPI. Add pre-commit hooks for prompt quality checks.

This document was automatically generated from the Prompt Toolkit v2 project. The project has been transformed from a simple Proof of Concept to a production-grade application with 49 files, multi-dimensional analysis, AI provider integration, desktop GUI, prompt library, and 69 passing tests.

End of Documentation